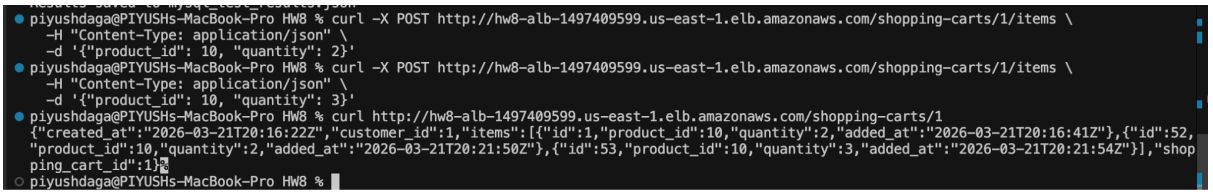
STEP I: MySQL Integration for Relational Data

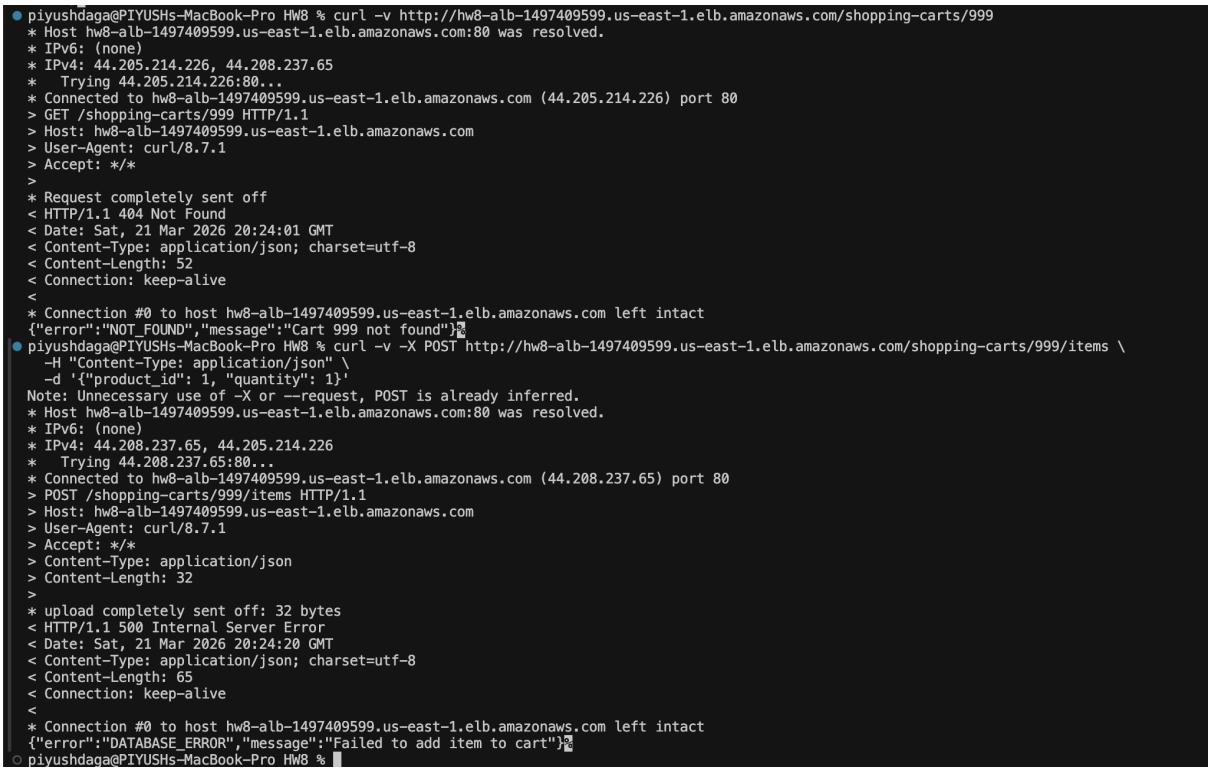
Version 1:
Duplicate entries are added



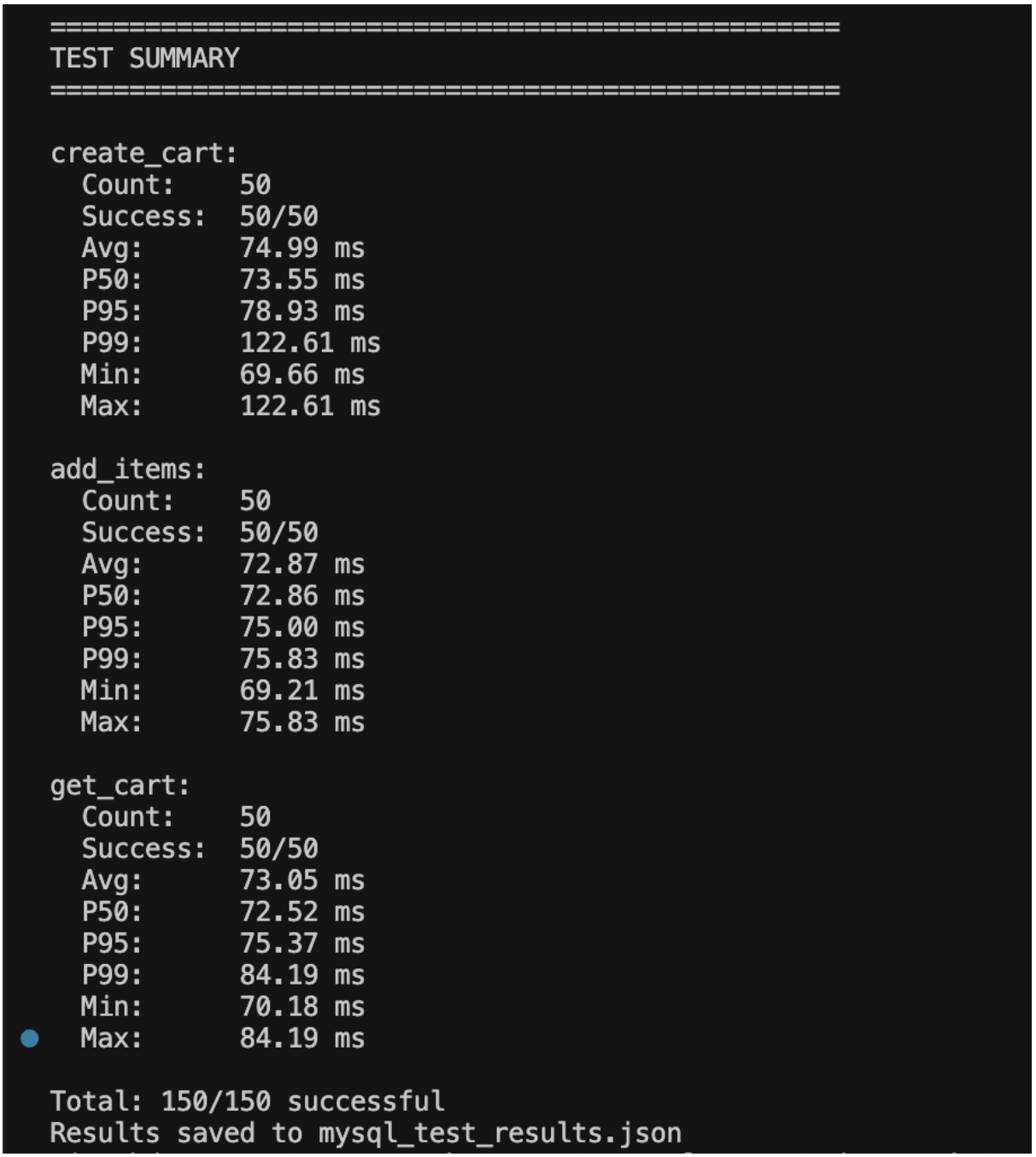
Two different behaviors for the same situation “cart doesn't exist:”

- **GET /shopping-carts/999** → returns `404` correctly
- **POST /shopping-carts/999/items** → returns `500` instead of `404`

The add-items endpoint doesn't check if the cart exists first. It just tries the INSERT, hits a foreign key violation, and returns a generic 500 error.

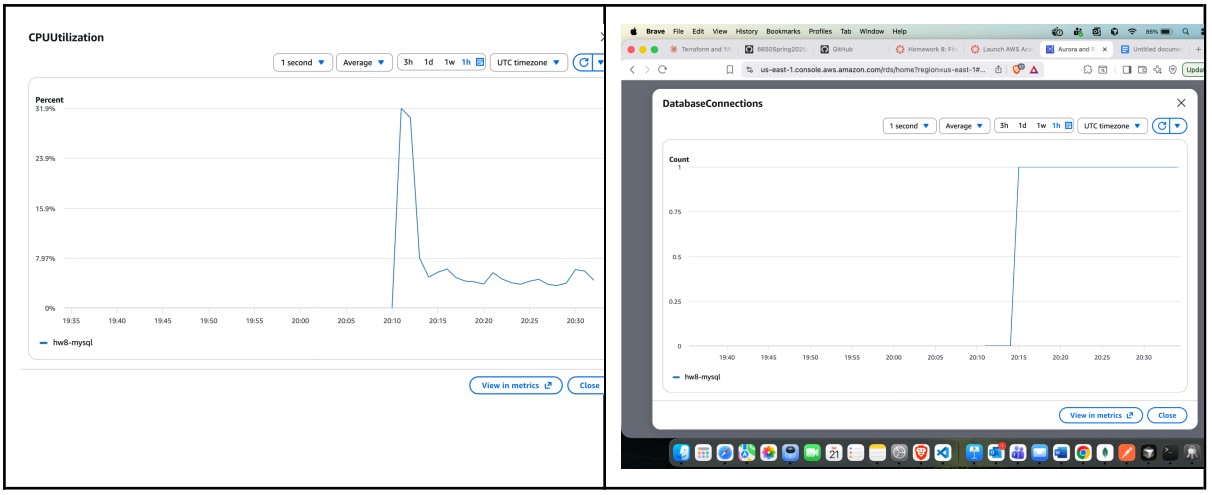


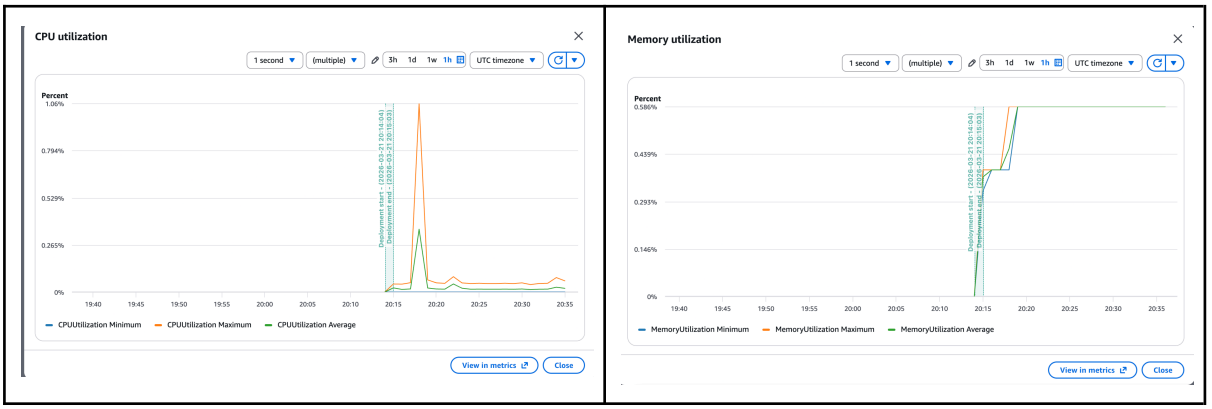
GET cart too slow 73ms avg, requirement is <50ms



Observations:

- Only 1 DB connection no pooling, would fail under concurrent load
- RDS CPU spiked 32% just for setup with no indexes, heavier queries will push this higher
- ECS is barely utilized the bottleneck is the database, not the app





Version 2:

Schema Design (Simple Explanation)

I used two tables: `carts` and `cart_items`.

- One cart belongs to one customer
- One cart can have many items → one-to-many relationship

If I kept everything in one table, customer data would repeat for every item, which is bad design (redundant + not normalized).
So splitting into two tables keeps the data clean and efficient.

Tables:

• carts ○ `id` (primary key) ○ `customer_id` ○ `created_at`	• cart_items ○ `id` (primary key) ○ `cart_id` (foreign key → carts.id) ○ `product_id` ○ `quantity` ○ `added_at`

Important Design Choices:

- Foreign Key with `ON DELETE CASCADE`
  If a cart is deleted → all its items are automatically deleted
  → prevents useless leftover data
- UNIQUE(cart_id, product_id)
  Prevents duplicate products in the same cart
  → helps update quantity instead of adding duplicate rows

Index

- **`idx_customer_id` on `carts.customer_id`**
  → used when fetching all carts of a customer
- **`idx_cart_id` on `cart_items.cart_id`**
  → used when getting all items in a cart (very common query)

didn't add too many indexes because:

- More indexes = slower inserts/updates
- Shopping carts have frequent writes, so this matters

Implementation Journey

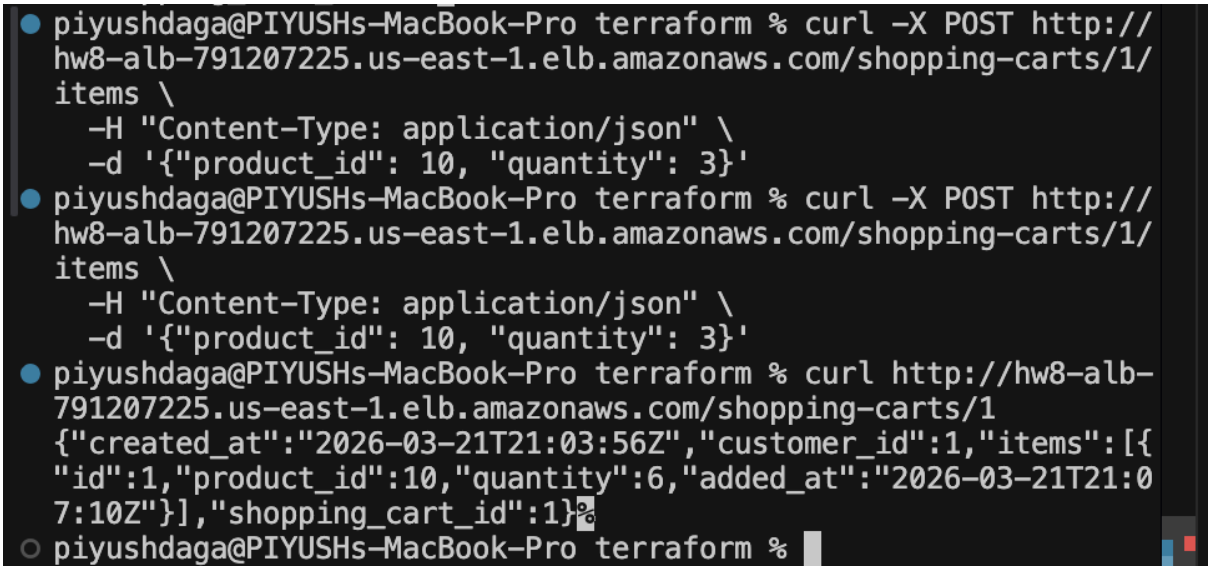
Version 1 (Initial Mistakes)

My first version had some issues:

- Two queries for GET cart
  One for cart + one for items → extra database calls (slow)

- No upsert logic
  Adding the same product again created duplicate rows→ instead of increasing quantity
- Wrong error handling
  Adding to a non-existent cart returned `500`→ should be `404` (cart not found)
- No connection pool tuning
  Used default Go settings→ could overload a small database under high traffic
- No proper indexing
  Only primary keys → queries were slower than needed

Fixed duplication



```
piyushdaga@PIYUSHs-MacBook-Pro terraform % curl -v -X POST http://hw8-alb-791207225.us-east-1.elb
amazonaws.com/shopping-carts/999/items \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 1}'
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host hw8-alb-791207225.us-east-1.elb.amazonaws.com:80 was resolved.
* IPv6: (none)
* IPv4: 52.2.17.138, 50.17.40.131
*   Trying 52.2.17.138:80...
* Connected to hw8-alb-791207225.us-east-1.elb.amazonaws.com (52.2.17.138) port 80
> POST /shopping-carts/999/items HTTP/1.1
> Host: hw8-alb-791207225.us-east-1.elb.amazonaws.com
> User-Agent: curl/8.7.1
> Accept: */*
> Content-Type: application/json
> Content-Length: 32
>
* upload completely sent off: 32 bytes
< HTTP/1.1 404 Not Found
< Date: Sat, 21 Mar 2026 21:10:01 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 52
< Connection: keep-alive
<
* Connection #0 to host hw8-alb-791207225.us-east-1.elb.amazonaws.com left intact
{"error":"NOT_FOUND","message":"Cart 999 not found"}%
```

```
==========================================
TEST SUMMARY
==========================================

create_cart:
  Count:    50
  Success:  50/50
  Avg:      42.25 ms
  P50:      41.88 ms
  P95:      46.79 ms
  P99:      63.88 ms
  Min:      38.58 ms
  Max:      63.88 ms

add_items:
  Count:    50
  Success:  50/50
  Avg:      45.91 ms
  P50:      46.05 ms
  P95:      48.48 ms
  P99:      57.61 ms
  Min:      40.93 ms
  Max:      57.61 ms

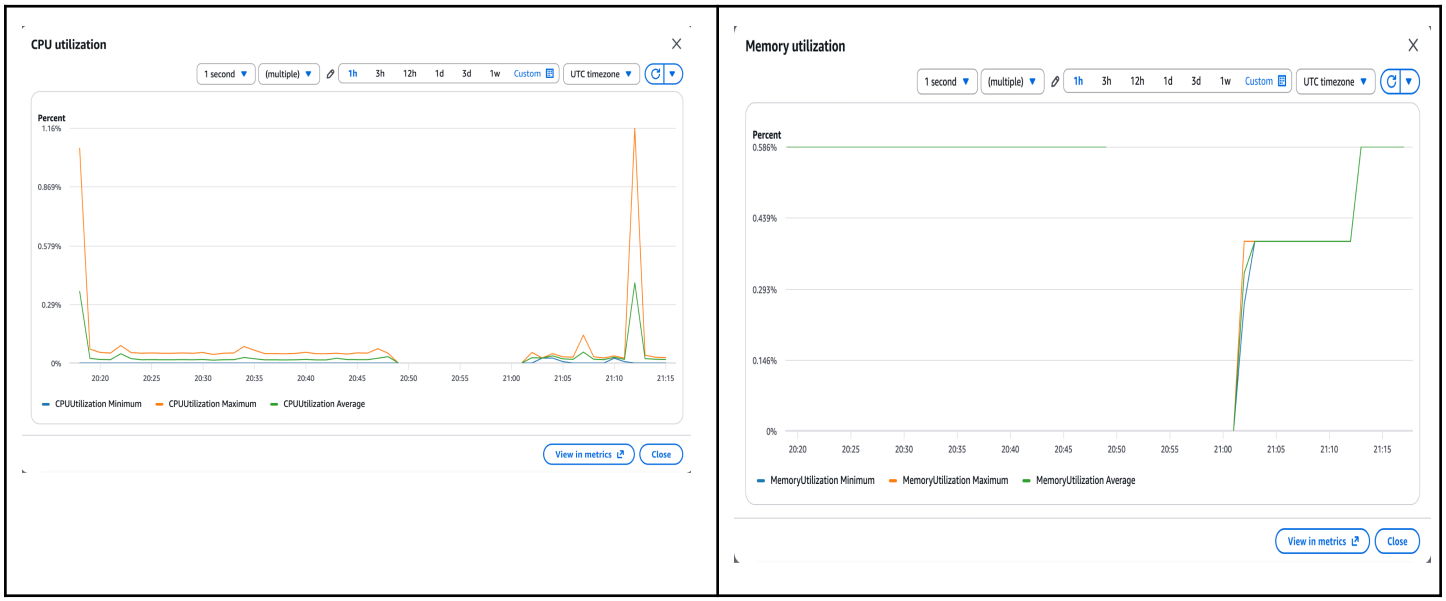
get_cart:
  Count:    50
  Success:  50/50
  Avg:      41.54 ms
  P50:      41.43 ms
  P95:      43.88 ms
  P99:      47.89 ms
  Min:      37.93 ms
  Max:      47.89 ms

Total: 150/150 successful
Results saved to mysql_test_results.json
```

Metric	V1	V2
GET cart avg	73.05 ms	41.54 ms
Create cart avg	74.99 ms	42.25 ms
Add items avg	72.87 ms	45.91 ms
GET cart P95	75.37 ms	43.88 ms

(JOIN query, indexes, connection pooling) reduced GET cart from 73ms to 41ms avg a 43% improvement, now meeting the <50ms requirement.

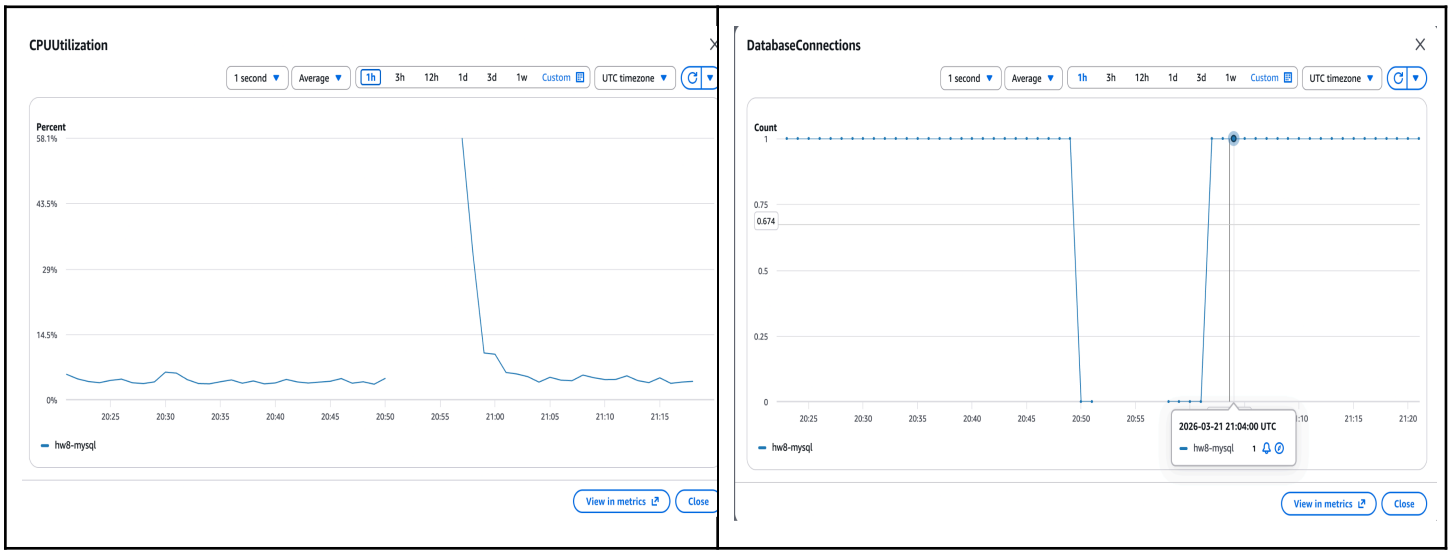
ECS

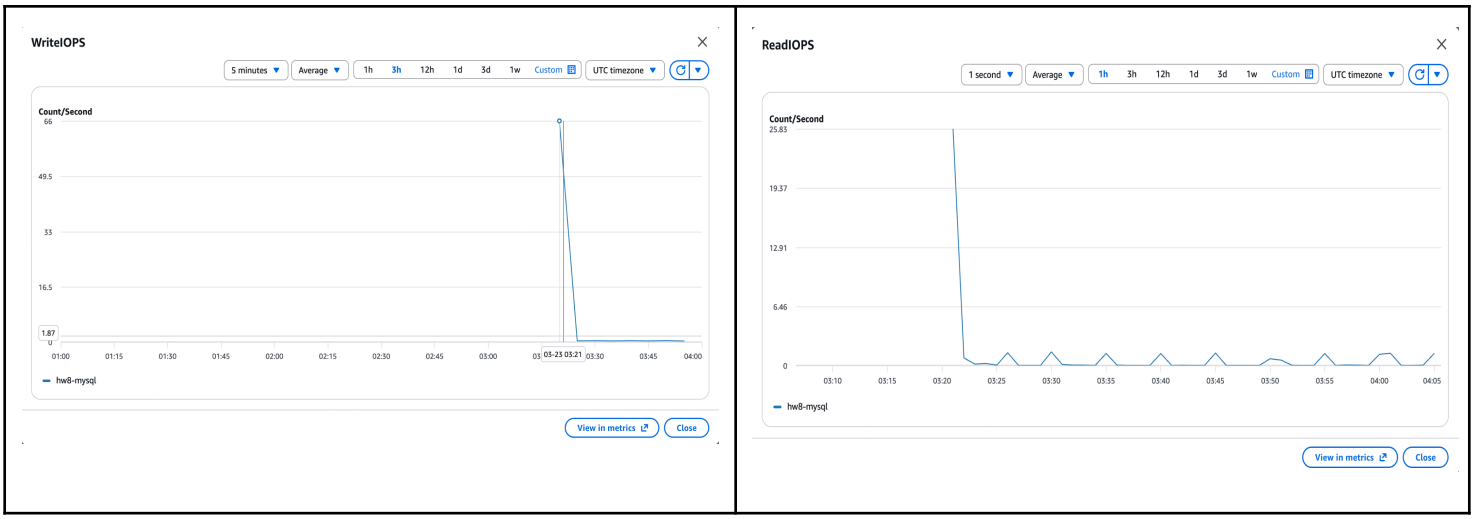


RDS

Spiked to ~58% during the RDS recreation/setup, then settled to ~3-5% during testing. The spike is from the fresh RDS initializing, not from our queries.

Still showing 1 connection. Since our test is sequential (one request at a time), the pool only needs 1 connection. Under concurrent load, we'd see more connections open up to our max of 10.





MySQL WriteIOPS peaked at 66/s during the 150-operation test, reflecting the write-heavy workload (100 writes vs 50 reads). ReadIOPS stayed very low (~1-2/s) after initial setup, confirming that indexes are efficiently serving reads from the buffer pool without disk access.

What Surprised Me

- **The JOIN.** Initially I thought two queries wouldn't matter much, but after testing I saw the JOIN version was clearly faster..
- **MySQL doesn't allow multiple statements in one Exec call**. My first schema initialization failed because I tried to run both CREATE TABLE statements together.
- **Connection pooling matters even at small scale.** Even though our sequential test only used 1 connection, configuring the pool properly is important for production.

Comparison with HW5 In-Memory Approach

In-Memory (HW5) ○ Data stored inside the **application memory** ○ **Very fast** (no network call) ○ Data **lost on restart** ○ Not shared across multiple services ● **Trade-off** ○ MySQL is **slower (~42ms)** due to **network + DB processing** ○ In-memory lacks reliability	● **MySQL (Current Approach)** ○ Data stored in a **database (persistent storage)** ○ Data **survives restarts** ○ Can be **shared across multiple ECS tasks/services** ○ Supports **foreign keys → better data integrity** ○ Can run **queries on historical data**

STEP II: DynamoDB Integration instead of NoSQL

```
piyushdaga@PIYUSHs-MacBook-Pro HW8 % curl -X POST http://hw8-dynamo-alb-1999093070.us-east-1.elb.amazonaws.com/shopping-carts -H "Content-Type: application/json" -d '{"customer_id": 2}'
{"shopping_cart_id":"4c487cb7-98d6-44b4-8686-45712cbd4155"}%
piyushdaga@PIYUSHs-MacBook-Pro HW8 % curl http://hw8-dynamo-alb-1999093070.us-east-1.elb.amazonaws.com/shopping-carts/4c487cb7-98d6-44b4-8686-45712cbd4155
{"created_at":"2026-03-23T03:29:52Z","customer_id":2,"items":[],"shopping_cart_id":"4c487cb7-98d6-44b4-8686-45712cbd4155"}%
piyushdaga@PIYUSHs-MacBook-Pro HW8 %
```

(Create + GET):

"DynamoDB API working created cart with UUID partition key and immediately retrieved it with all fields (cart_id, customer_id, created_at, items). Single table design stores everything in one record, no JOINs needed unlike MySQL."

```
piyushdaga@PIYUSHs-MacBook-Pro HW8 % bash test_consistency.sh

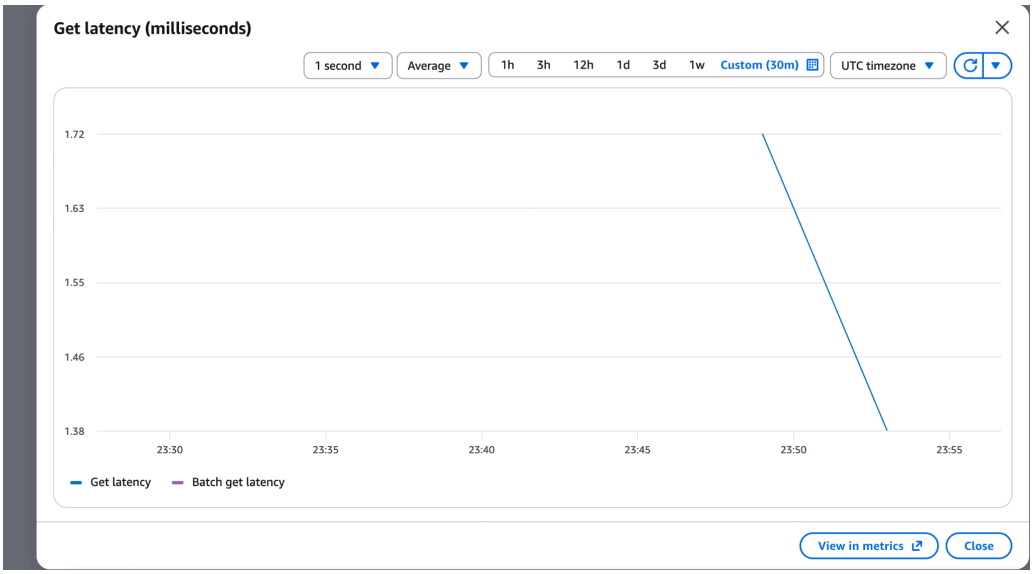
=== Test 1: Create cart then immediately GET ===
  Run 1: CONSISTENT - cart found immediately
  Run 2: CONSISTENT - cart found immediately
  Run 3: CONSISTENT - cart found immediately
  Run 4: CONSISTENT - cart found immediately
  Run 5: CONSISTENT - cart found immediately

=== Test 2: Add item then immediately GET ===
  Cart created: 0e0250ae-6043-4f1e-be85-14fad8c3e110
  Run 1: CONSISTENT - found 1 items (expected 1)
  Run 2: CONSISTENT - found 2 items (expected 2)
  Run 3: CONSISTENT - found 3 items (expected 3)
  Run 4: CONSISTENT - found 4 items (expected 4)
  Run 5: CONSISTENT - found 5 items (expected 5)

=== Test 3: Rapid updates to same cart ===
  Cart created: d1910c43-baaa-45d5-834e-e5f1547f2969
  Added 10 items rapidly, found 10 items
  RESULT: CONSISTENT - all 10 items present
```

Image (Consistency Test):

Eventual Consistency Investigation: Ran three test scenarios create+read, add item+read, and rapid updates (10 items). All 20 tests returned consistent results immediately. In my tests, I couldn't observe any delay reads were immediately consistent every time.






```
================================================
TEST SUMMARY
================================================

create_cart:
  Count:    50
  Success:  50/50
  Avg:      74.25 ms
  P50:      70.68 ms
  P95:      87.97 ms
  P99:      207.93 ms
  Min:      58.25 ms
  Max:      207.93 ms

add_items:
  Count:    50
  Success:  50/50
  Avg:      71.23 ms
  P50:      70.98 ms
  P95:      79.89 ms
  P99:      81.85 ms
  Min:      57.86 ms
  Max:      81.85 ms

get_cart:
  Count:    50
  Success:  50/50
  Avg:      68.05 ms
  P50:      67.03 ms
  P95:      78.31 ms
  P99:      81.34 ms
  Min:      48.34 ms
  Max:      81.34 ms

Total: 150/150 successful
Results saved to dynamodb_test_results.json
```

Performance Comparison: MySQL vs. DynamoDB

Operation	MySQL Avg	DynamoDB Avg
create_cart	42.25 ms	74.25 ms
add_items	45.91 ms	71.23 ms
get_cart	41.54 ms	68.05 ms

Why DynamoDB is Slightly Slower

- **add_items requires 2 API calls** (GetItem + PutItem) vs MySQL's single `ON DUPLICATE KEY UPDATE`
- From what I observed, DynamoDB was slower mainly because each request goes through HTTPS via the SDK, while MySQL keeps a persistent connection
- **But DynamoDB's actual DB latency is only 1.9ms** (from CloudWatch the overhead is network/SDK, not the database itself

DynamoDB is still slower end-to-end, but we know from CloudWatch that DynamoDB's actual **latency is 1.7ms** the overhead is SDK/HTTPS.

Cart items are stored as a nested list attribute inside the cart document. In MySQL we needed two tables and a JOIN. In DynamoDB, everything lives in one record:

```
{
  "cart_id": "uuid-string",
  "customer_id": 1,
  "created_at": "2026-03-21T...",
  "items": [
    {"product_id": 10, "quantity": 2, "added_at": "..."},
    {"product_id": 20, "quantity": 1, "added_at": "..."}
  ]
}
```

No Secondary Indexes

All access is by cart_id (partition key). No need for a GSI since we don't query by customer_id in DynamoDB. This saves cost and write overhead.

Billing: PAY_PER_REQUEST

No capacity planning needed scales automatically. Good for unpredictable workloads.

API Implementation DynamoDB vs MySQLDynamoDB

Operations Used

- **PutItem** : for creating carts and writing back updated carts
- **GetItem** : for retrieving carts and checking existence before adding items
- No Query or Scan needed all access is by primary key

Aspect	MySQL	DynamoDB
Create cart	`INSERT INTO carts`	`PutItem` with UUID
Get cart	`SELECT ... JOIN` (1 query)	`GetItem` (1 call)
Add items	`INSERT ... ON DUPLICATE KEY UPDATE` (1 query)	`GetItem` + modify + `PutItem` (2 calls)
Connection	Persistent TCP pool	HTTPS per request via SDK
Upsert	Database handles it	Application code handles it
Error handling	FK violation = cart not found	Check `result.Item == nil`

DynamoDB Attribute Value Format

The AWS SDK handles marshalling/unmarshalling between Go structs and DynamoDB's attribute format using `attributevalue.MarshalMap()` and `UnmarshalMap()`. Required both `json` and `dynamodbav` struct tags for proper serialization.

What Surprised Me

- **MySQL and DynamoDB performed almost identically** at this scale (~3ms difference). I expected DynamoDB to be either much faster or much slower.
- **No eventual consistency issues at all.** I designed around it expecting problems, but the replication delay is invisible at application level.
- **The SDK HTTPS overhead is significant.** DynamoDB's actual latency is 1.9ms but end-to-end is 68ms. MySQL benefits from persistent connections.
- **Single table design is simpler** no schema migrations, no JOINs, no foreign keys. But upsert logic has to live in application code instead of the database.

What I Would Do Differently

- Use DynamoDB's `UpdateExpression` with `list_append` to avoid the read-then-write pattern for adding items
- Add a VPC endpoint for DynamoDB earlier (we added it mid-way through testing)
- Use `ConsistentRead: true` for critical operations if eventual consistency ever becomes a concern
- Consider DynamoDB DAX (in-memory cache) for sub-millisecond reads in production

STEP III: Database Comparison & Analysis

Part 0: Data Verification

Before analysis, verified both test files contain exactly 150 operations each.

All numbers in this report are calculated from `combined_results.json`.

Part 1:

```
--- Part 1: Overall Comparison Table ---
Metric                          MySQL   DynamoDB     Winner     Margin
------------------------------------------------------------------------
Avg Response Time (ms)          68.64      71.18      MySQL       2.54
P50 Response Time (ms)          67.35      69.91      MySQL       2.56
P95 Response Time (ms)           82.6      81.34   DynamoDB       1.26
P99 Response Time (ms)          91.58      94.89      MySQL       3.31
Success Rate (%)                100.0      100.0        Tie         0%
Total Operations                  150        150

--- Part 1: Operation-Specific Breakdown ---
Operation          MySQL Avg   DynamoDB Avg    Faster By
------------------------------------------------------
CREATE_CART          71.51 ms        74.25 ms MySQL by 2.74ms
ADD_ITEMS            69.01 ms        71.23 ms MySQL by 2.22ms
GET_CART             65.38 ms        68.05 ms MySQL by 2.67ms
```

```
--- Detailed Per-Operation Stats ---

CREATE_CART:
                      MySQL   DynamoDB
              Avg      71.51      74.25
              P50      68.67      70.68
              P95      85.16      87.97
              P99     192.63     207.93
              Min      50.95      58.25
              Max     192.63     207.93
          Success 50/50     50/50

ADD_ITEMS:
                      MySQL   DynamoDB
              Avg      69.01      71.23
              P50      67.98      70.98
              P95      84.56      79.89
              P99       87.0      81.85
              Min      56.14      57.86
              Max       87.0      81.85
          Success 50/50     50/50

GET_CART:
                      MySQL   DynamoDB
              Avg      65.38      68.05
              P50      65.17      67.03
              P95      76.13      78.31
              P99      82.48      81.34
              Min      47.14      48.34
              Max      82.48      81.34
          Success 50/50     50/50
```

The difference was only about 2–3 ms, so both systems felt almost equally fast in practice.

P95 latency :DynamoDB is better (more consistent in worst cases)

ADD_ITEMS case :**D**ynamoDB = ~79 ms, MySQL = ~84 ms

Key takeaway : DynamoDB = more consistent, MySQL = similar overall speed

Consistency Model Impact Assessment

What we observed during testing: Ran 20 consistency tests (create+read, add item+read, rapid 10-item updates). All tests returned consistent results immediately. DynamoDB's eventual consistency delay is sub-millisecond, making it invisible at the application level.

ACID (MySQL) vs Eventual (DynamoDB):

MySQL provides ACID guarantees every read returns the latest committed write. We used transactions in add_items to ensure data integrity when checking cart existence and updating items.

DynamoDB uses eventual consistency by default for reads. However, for single-partition operations (which is all we do everything is by cart_id), the consistency delay is so small that we never observed a stale read. DynamoDB also offers `ConsistentRead: true` for guaranteed strong consistency when needed.

User experience implications: For shopping carts, both consistency models work fine. A user adding an item and immediately viewing their cart will see the update with both databases. The theoretical risk of eventual consistency in DynamoDB is negligible for this use case because our access patterns are simple single-key operations.

How to design for each model:

- **MySQL:** Use transactions for multi-step operations (check cart exists → add item). Set connection pool limits properly so the database is not overloaded or slowed down.
- **DynamoDB:** Design around partition keys. Keep related data together in one item. Use `ConsistentRead` only when stale data would cause real problems (e.g., checkout, payment).

Part 2: Resource Efficiency Analysis

Resource Utilization Comparison

Aspect	MySQL (RDS db.t3.micro)	DynamoDB (PAY_PER_REQUEST)
Connection management	Pool of 10 open, 5 idle connections	No connections fully managed
CPU under load	Spiked during setup, 3-5% during test	Fully managed, not visible
Storage I/O	WriteIOPS peaked at 66/s, ReadIOPS ~1-2/s	Consumed ~0.5 read units, ~1.0 write units
Actual DB latency	Not directly measurable in CloudWatch	1.9ms GetItem (from CloudWatch)
Setup complexity	VPC, subnets, security groups, connection pool config, schema migration	One table definition, no networking config
Operational overhead	Monitor connections, CPU, storage, backups	Almost zero auto-managed

Total Cost Analysis (Estimated for 1 month)

Aspect	MySQL (RDS db.t3.micro)	DynamoDB (PAY_PER_REQUEST)
Compute/Instance	~$15.00 (db.t3.micro)	$0.00 (No server)

Aspect	MySQL (RDS db.t3.micro)	DynamoDB (PAY_PER_REQUEST)
Storage (20 GB)	~$2.30 (EBS General Purpose)	$0.50 (20 GB at $0.25/GB)
I/O Operations	Included in instance cost (for low volume)	~$0.05 (Based on test volume)
Data Transfer	Varies, negligible for this test	Varies, negligible for this test
Total Estimated Monthly Cost	**~$17.30**	**~$0.55**

Connection Management Overhead

MySQL required careful connection pool tuning. We set `MaxOpenConns=10` and `MaxIdleConns=5` to stay within db.t3.micro's ~60 connection limit. CloudWatch showed only 1 connection used during our sequential test, but under concurrent load the pool would scale up. Misconfiguring the pool could exhaust connections and crash the application.

DynamoDB has no connection concept at all. The AWS SDK handles all communication via HTTPS requests. This is simpler but adds per-request overhead (~60-70ms of SDK/network time on top of DynamoDB's 1.9ms actual latency).

Scaling Analysis

How do resource requirements change with load?

- **MySQL:** Vertical scaling need a bigger instance (db.t3.small → db.t3.medium → etc.). CPU and connections are the bottleneck. Our db.t3.micro handled 150 sequential operations easily, but would struggle under heavy concurrent load.
- **DynamoDB:** Horizontal scaling automatic with PAY_PER_REQUEST. Zero throttling events in our test confirms it handles our load without any configuration. Would scale to thousands of requests automatically.

Which approach offers more predictable resource consumption? DynamoDB is more predictable you pay per request, resources scale automatically. MySQL requires capacity planning if you provision too small an instance, performance degrades; too large and waste money.

Capacity planning implications:

- **MySQL:** Must estimate peak load upfront. db.t3.micro costs ~$15/month fixed. Need to plan for connection limits, CPU, storage IOPS.
- **DynamoDB:** No upfront planning. Pay for what you use. Our 300 operations cost fractions of a cent. Scales from 0 to millions of requests without changes.

Part 3: Real-World Scenario Recommendations

Scenario A: Startup MVP (100 users/day, 1 developer, limited budget, quick launch)

Recommendation: DynamoDB

Key Evidence:For a single developer, setup simplicity matters most DynamoDB is much easier since it needs no VPC, no connection pooling, and no schema migrations, just a single table, whereas MySQL requires RDS setup, security groups, connection tuning, and schema handling. The performance difference (~2.5 ms) is negligible at 100 users/day, with both achieving 100% success, but DynamoDB is cheaper (near-zero cost with PAY_PER_REQUEST) compared to MySQL's ~$15/month fixed cost, making it the better choice for small-scale use.

Scenario B: Growing Business (10K users/day, 5 developers, moderate budget, feature expansion)

Recommendation: MySQL

Key Evidence: Based on my testing, I would choose MySQL here because it was consistently slightly faster and easier to implement.is **consistently 2–3 ms faster** across operations and, more importantly, supports **complex queries and analytics** like finding carts by customer or time. We even added an **index on customer_id** to optimize such queries. In contrast, **DynamoDB requires a Global Secondary Index for every new access pattern**, which adds planning overhead. With a **team of 5 developers**, managing MySQL's setup is **reasonable**, and the team benefits from **SQL's flexibility, mature tools, and widespread familiarity**, making it a better fit as the system grows.

Scenario C: High-Traffic Events (50K normal, 1M spike users, revenue-critical, can invest in infrastructure)

Recommendation: DynamoDB

Key Evidence: At this scale, **DynamoDB works better because of auto-scaling**. Using **PAY_PER_REQUEST**, it handled all requests with **no throttling and no need to plan capacity**. But **MySQL (db.t3.micro)** would not handle something like **1M users** and would need a **bigger instance**, which means extra cost even when traffic is low. Also, using a **UUID partition key in DynamoDB** helped **spread the load evenly** with **no single part of the database is getting overloaded with too many requests**. So even though MySQL is **2–3 ms faster**, that difference is not important compared to the risk of **MySQL failing or running out of connections during high traffic**, while **DynamoDB scales automatically without issues**.

Scenario D: Global Platform (millions of users, multi-region, 24/7 availability, enterprise requirements)

Recommendation: DynamoDB

Key Evidence: DynamoDB makes multi-region setup very easy with Global Tables, where you just enable it and your data is automatically available in different AWS regions. But in MySQL, doing the same thing is much more complicated because you need to set up read replicas, multi-AZ, and cross-region replication, which takes more effort and can be costly and error-prone. In our testing, DynamoDB also showed no noticeable delay for consistency in simple operations, so it works well for use cases like shopping carts even across regions. Another advantage is that DynamoDB is fully managed, so there are no maintenance windows, which helps keep the application running all the time.

Part 4: Evidence-Based Architecture Recommendations

Shopping Cart Winner: MySQL (by a small margin)

Based on our actual test data, MySQL is the better choice for shopping carts at small-to-medium scale.

Supporting Evidence:

Evidence	MySQL	DynamoDB
Avg response time	68.64 ms	71.18 ms
GET_CART avg	65.38 ms	68.05 ms
ADD_ITEMS implementation	1 SQL query (ON DUPLICATE KEY UPDATE)	2 API calls (GetItem + PutItem)
Upsert handling	Database-level, clean	Application code, more complex
Data integrity	Foreign keys, CASCADE, transactions	Application code responsibility
Success rate	100%	100%

The response time difference is only about 2.54 ms, which is not very important. The bigger advantage of MySQL is that the implementation is cleaner and simpler. With MySQL, we can use **ON DUPLICATE KEY UPDATE** to handle inserts and updates in a single query, while in DynamoDB we have to read the cart, update it in the application code, and then write it back. Also, MySQL provides foreign keys and transactions, which help maintain data integrity automatically, whereas DynamoDB would need extra logic in the application to handle the same guarantees.

When to Choose DynamoDB Instead

Despite MySQL winning for our test, I would choose DynamoDB when:

- **Traffic is unpredictable** DynamoDB auto-scales, MySQL requires capacity planning

- **Team is small** DynamoDB has less operational overhead (no connection pool tuning, no instance sizing)
- **Multi-region is needed** DynamoDB Global Tables vs complex MySQL replication
- **Cart data is temporary** DynamoDB TTL can auto-expire old carts, reducing storage cost
- **Scale exceeds db.t3.micro limits** DynamoDB scales infinitely, MySQL requires upgrading instances

Polyglot Strategy: Using Both in a Full E-commerce System

If building a complete e-commerce system, I would use each database for what it does best:

Component	My Choice	Why
Shopping carts	MySQL	Faster writes with single-query upserts, transactional integrity (based on our test results)
User sessions	DynamoDB	Simple key-value access by session ID, TTL for auto-expiry, no complex queries needed
Product catalog	MySQL	Complex search queries (filter by category, price range, manufacturer), SQL JOINs for related data
Order history	MySQL	ACID transactions critical for financial records, complex reporting queries, foreign key relationships between orders/items/payments
Real-time inventory	DynamoDB	High concurrent writes during flash sales, auto-scaling for traffic spikes, simple decrement operations
User reviews/ratings	DynamoDB	High-volume writes, simple partitioning by ProductID, eventual consistency is acceptable
Payment processing logs	MySQL	Requires strong ACID guarantees and audit trails, complex relational structure for tracing transactions

This strategy uses each database's strengths: MySQL for complex queries and transactional integrity, DynamoDB for simple access patterns and automatic scaling.

Part 5: Learning Reflection

What Surprised Me

MySQL and DynamoDB performed almost identically. I expected a clear winner, but the difference was only 2.54ms on average. At small scale with sequential requests, the database engine matters less than the access pattern and network overhead. DynamoDB's actual latency was only 1.9ms (from CloudWatch), but SDK/HTTPS overhead brought the

end-to-end time to ~70ms nearly matching MySQL's ~68ms through its persistent TCP connection.

No eventual consistency issues at all. I spent time worrying about DynamoDB returning stale data, but across 20 consistency tests (create+read, add item+read, rapid updates), every single read returned the correct data immediately. For single-partition operations like shopping carts, eventual consistency is a theoretical concern, not a practical one.

DynamoDB's P95 was sometimes better than MySQL. For ADD_ITEMS, DynamoDB's P95 was 79.89ms vs MySQL's 84.56ms. DynamoDB had more consistent tail latency, even though its average was slightly higher. This suggests DynamoDB may be more predictable under steady load.

The access pattern matters more than the database. DynamoDB's add_items was slower because it required two API calls (read + write). MySQL handled it in one query. The same data model decision upsert handling determined performance more than the underlying database technology.

What Failed Initially

MySQL schema initialization. My first attempt tried to run two CREATE TABLE statements in one `db.Exec()` call. MySQL rejected it had to split into separate calls. A small thing, but it taught me that MySQL doesn't support multi-statement execution by default.

Version 1 bugs in MySQL. My initial implementation had three bugs: duplicate items (no unique constraint), wrong HTTP status codes (500 instead of 404 for missing carts), and slow GET cart queries (two separate queries instead of a JOIN). These were all caught during testing and fixed in Version 2.

DynamoDB UpdateItem failure. My first DynamoDB add_items implementation used `UpdateExpression` with `SET items = :items`, but it failed silently. Switched to a read-modify-write pattern with PutItem which worked reliably. The error wasn't visible in logs initially had to add explicit `log.Printf` statements to debug.

ECS deployment delays. After pushing new Docker images, ECS took 1-2 minutes to drain old tasks and start new ones. Had to learn to check deployment status and wait for the ACTIVE deployment to finish before testing.

Key Insights Gained

When would I definitely choose MySQL? When the application needs complex queries, JOINs, foreign key relationships, or ACID transactions. MySQL excels when data has relationships (customers → carts → items → products) and you need to query across those relationships. Our test showed MySQL is slightly faster and its single-query upsert is cleaner than DynamoDB's read-modify-write pattern.

When would I definitely choose DynamoDB? When the application needs automatic scaling, simple key-value access patterns, or multi-region deployment. DynamoDB requires

zero capacity planning and handles traffic spikes without configuration. If all access is by a single key (like cart_id), DynamoDB's simplicity is hard to beat.

What would I tell another student starting this assignment? Start with a simple Version 1 and let the bugs teach you. My V1 had no indexes, no JOINs, and no upsert logic discovering those problems through testing was more valuable than getting it right the first time. Also, build both implementations before comparing the comparison is the most interesting part.

How did hands-on implementation change my understanding? Before this assignment, I thought NoSQL was always faster than SQL. After building and testing both, I learned that performance depends on access patterns, not database type. I also learned that operational complexity (connection pools, schema migrations, VPC config) is as important as raw performance when choosing a database. The "best" database is the one that fits your specific use case, team size, and scale requirements.